PROJECT AND TEAM INFORMATION

Algorithmic Trading and Market Simulation Platform

Student / Team Information

Team Name: ***Team ID:***	***TradeSim*** ***DSCPP-III-2026-T331***
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	***Jatrana, Satvik –*** ***2510019213*** ***2510019213@geu.ac.in*** 
Team member 2 *(Last Name, name: student ID: email, picture):*	***Bhatt, Siddharth –*** ***2510012106*** ***2510012106@geu.ac.in*** 
Team member 3 *(Last Name, name: student ID: email, picture):*	***Uniyal, Akshat –*** ***2510370489*** ***2510370489@geu.ac.in*** 
Team member 4 *(Last Name, name: student ID: email, picture):*	***Kumar, Sourabh –*** *student ID* *email ID* picture

PROPOSAL DESCRIPTION

Motivation

Stock Market trading applications get lots of buy and sell orders on daily basis and now they should keep track of them and process them. One of the major questions is that in which order to be handled and how to match the buy order with a sell order.

In this application ,the orders arent to be simply displayed but also be arranged based upon rules and also when the right match found then the trade should be executed. This process involves algorithms used for searching, comparing and updating the orders.

The trading applications provides the final outcome of the trade to the users , without providing much information about the process.
The TradeSim will show a small simulated version of this process and the virtual trader will be able to submit the buy or sell order,then the system will processand place the order in the order book accordingly.Hence, The matching engine will look for a match to the order that to be executed.
The TradeSim will use the concepts of Data Structures in C and OOPs with C++ in one application itself instead of studying them differently. It also has scope for the addition of other features like historical data and strategy testing.

State of the Art / Current solution

In Actual financial markets, electronic trading systems take in buy and sell orders and act upon them in accordance with predetermined rules. The orders are kept in what is known as an order book, in which both price and time are important when determining which order takes precedence. A matching system then searches for buy and sell orders that are compatible and produces trades when the necesary conditions have been met.

Traders usually use stock exchanges or brokerage platforms to interact with these systems, while the internal process of order management and matching is kept hidden from them.
Educational trading programs usually include features like virtual portfolios, price tracking, transaction histories, and the ability to carry out Simulated buys or sells. while such applications can help with learning the basics of trading, they do not always clearly illustrate the data structures and algorithms that are at work in the trading process.

TradeSim is concerned with this fundamental process. Rather than linking to an actual exchange, the project will set up a controlled simulation with the use of C/C++. This will enable us to show how orders are stored, prioritised, matched, recorded, and reflected in the virtual portfolios.

Project Goals and Milestones

General Goals:

- *Create a stock market simulation using C/C++ that allow its users to place and execute the buy and sell orders.*
- *Have an organized order book which keep information about the executed and pending orders.*
- *Build a matching engine that matches the orders and handles their execution.*
- *Track users(virtual traders) and their portfolios by logging thier trades and funds.*
- *Use the appropriate data structures and algorithms to store, search,and process orders.*
- *Use C++ object-oriented principles for distinct parts like Order,Trader,OrderBook, Matching Engine,trade and Portfolio.*
- *Store important informations like orders,trades and portfolios using file based storage.*
- *Test the system with various scenarios including normal, invalid,cancellation, full-fill,and partial-fill cases.*
- *Design the system in such a way that makes it easy to add new features like historical market data, trading strategies, quantitative analysis, and backtesting in the future.*

Milestones:

1. ***Requirement Analysis and C Data Structures (Weeks 1–2):*** *Finish figuring out the project requirements and how the trading process will work.Create the Order structure and build the basic data structures needed to store and manage orders.Make the first version of the console interface.*
2. ***Order Book and Order Management(Week 3):*** *Make separate buy and sell order books. Add features like inserting orders, canceling them, checking if they are valid, and handling priority based on price and time.*
3. ***Matching Engine (Week 4):*** *The matching logic will be connected to the order book. The engine will compare the highest-priority buy and sell orders and execute them when their prices enable a match. Cases where only a part of an order can be executed will also be addressed.*
4. ***C++ OOP Layer and Class Integration(Week 5):*** *The main parts of the program will be organized into C++ classes. These will include Order, Trader, OrderBook, MatchingEngine, Trade, and Portfolio. We will apply encapsulation, abstraction, constructors, and appropriate access control while connecting the classes.*
5. ***Trade and Portfolio Management(Week 6):*** *After a successful match, the resulting trade will be recorded and two traders' portfolios will be updated. Virtual cash and holdings will be adjusted and basic profit and loss calculations will be added.*
6. ***File Storage and Testing(Week 7):*** *File handling will be added for storing important orders, trades, and portfolio information. The system will be tested for various inputs, including incorrect orders, cancellations, complete executions, partial executions, and boundary cases.*
7. ***Integration, Demonstration and Documentation:(Final Phase):*** *All modules will be connected into the final application. We will run complete trading scenarios from order submission to portfolio update and record the test results. Screenshots will be prepared, document the data structures and algorithms used, and complete the final presentation.*

Project Approach

The project will be built as a modular C/C++ application. C is used to implement basic data structures and algorithms,While C++ organizes the whole project using object-oriented programming. This project includes features like Order, Trader, Order Book, Matching Engine, Trade,and Portfolio.

A trader starts by placing a buy or sell order , The order management part checks the input and adds the order to the right side of the order book.
The matching engine will matches the appropriate buy and sell orders. When the prices are matched,then the trade is executed. Here, the partial execution is also possible if the price matched but the quantity of the matched order is lesser than the original order. Each executed trade must be recorded and also be reflected in the trader's virtual portfolio.

The files will be used to store all the simulation data. The proposed solution will be implemented in C / C ++ programming language with the utilization of the corresponding Standard Template Library, C / C ++ compiler and IDE tools, and version control with Git.
The development process will follow the iterative approach, which will encompass design, implementation, testing/debugging, and integration phases. The testing will include validation of correct and incorrect inputs, order cancellation, price/time priority, completely filled orders, and partially filled orders.

Project Outcome / Deliverables

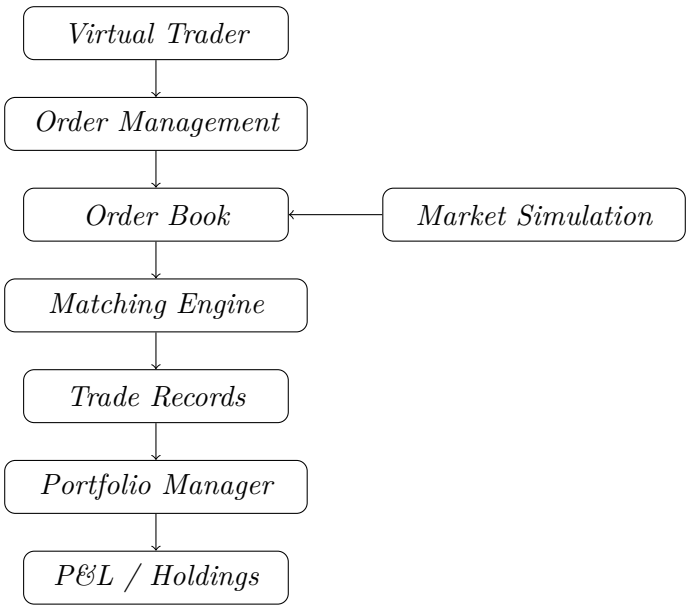
The main result of the project will be a working C/C++ market simulation where virtual traders can place buy and sell orders and see how these orders are handled.
The finished system will keep track of an order book, match orders that can be filled, record completed trades,and update virtual portfolios.
It will show how data structures and algorithms can be used to manage and process trading orders, and C++ object-oriented programming will be used to build the d ifferent parts of the system.

The main deliverables will include:

1. *TradeSim , a working application with virtual trading features.*
2. *An order book and matching engine which will be able to fully or partially execute trade orders.*
3. *Implementation of data structures and algorithms with proper documentation.*
4. *Records of the trades and portfolios including profits and loss.*
5. *Test cases and test results for various trading scenarios.*
6. *The source code, project report and presentation.*

The system will also form a basis for further works including historical market data, trading strategies, backtesting and quantitative analysis.

System Architecture (High Level Diagram)



High-Level Architecture of the TradeSim Market Simulation

The system begins with virtual trader placing a buy or sell order.The order-management module checks if the order is valid and sends it to the right side of the order book.The order book keeps track of all pending orders in a way that prioritizes them properly.

The matching engine will look for the best appropriate buy and sell orders that can be matched and executed. Once the appropriate condition has been satisfied, the possible amount is executed and the trade takes place. The trade is recorded and the accounts of the traders involved are updated.
The market simulation module can create and control orders and different trading scenarios which also allows the system to be tested in various conditions.

Assumptions

- *The project will be a simulated stock market with virtual money and has nothing connected to a real stock exchange market.*
- *The orders will be created by the virtual traders and through controlled market simulation .*
- *The project will focus on the core order-management and matching process of the real trading system.*
- *File based storage will be used at first to save important orders, trades, and portfolio records.*
- *Historical data, trading strategies, and backtesting will be future extensions, not part of the initial PBL implementation.*

Subject Integration

Data Structures in C:

- ***Structures:****Used to represent an order by storing information such as orderID, trader ID,price, quantity, order type,and timestamp.*
- ***Arrays:*** *Used where there is a need for structures that hold a fixed number of orders, traders, or records in memory.*
- ***Linked Lists:****Used to manage the dynamic data structures of the pending orders which are either added or deleted while simulating the system.*
- ***Queues:*** *Used to maintain temporal priority of those orders which are placed at the same price level.*
- ***Searching and Sorting:*** *Used to search for particular orders and to manage the buy and sell orders in accordance with the order book's priority.*

OOPs with C++:

- ***Classes and Objects:****Used to model the core components of the order management system including the Order , Trader, OrderBook, Matching Engine, Trade, and Portfolio.*
- ***Encapsulation:****Used to ensure that each entity encapsulates its state and behavior thus controlling how the data is accessed.*
- ***Abstraction:*** *Used to provide abstraction of the implementations and expose only key operations and services such as order submission, matching, and managing a trader's portfolio.*
- ***Constructors and Destructors:*** *Used to construct class objects by allocating memory and other necessary resources and also to destroy the objects by releasing memory and other resources.*
- ***Exception Handling:****Used to handle invalid inputs and incorrect orders without causing the complete application to terminate unexpectedly.*

Integration:
These two subjects work together in the main process of TradeSim. Data structures handle how orders are stored, arranged, and handled, while C++ classes break these tasks into separate and reusable parts. Together they cover the whole process, starting from when an order is submitted, managing the order book, matching orders, recording trades, and updating portfolios.

References

1. *cppreference.com, C++ Reference.*
   https://en.cppreference.com/
2. *cppreference.com, C Language Reference.*
   https://en.cppreference.com/w/c
3. *cppreference.com, C++ Standard Library Containers.*
   https://en.cppreference.com/w/cpp/container
4. *Overleaf Documentation.*
   https://www.overleaf.com/learn
5. *Course materials and lecture notes for Data Structures in C and OOPs with C++.*